SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: **Artificial Intelligence** **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. *(BL3)*

Assessment Tool Weightage: 50%

QUESTIONS: 5 **Duration : 1 Hour**

Total Marks:50

Annexure A

Analytical Problem Solving

Code of Conduct:

I SUBISH A G (192312139) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *Subish A G*

Annexure A

Analytical Problem Solving

1. Solve the Water Jug problem: you are given 2 jugs, a 4-gallon one and 3-gallon one. Neither has any measuring maker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into 4-gallon jug? Explicit assumptions: A jug can be filled from the pump, water can be poured out of a jug onto the ground, water can be poured from one jug to another and that there are no other measuring devices available.

2. Find out how Mars Rover, analyse and explore the samples on Mars by answering the following questions.
   i. What are the percept's for this agent?
   ii. Characterize the operating environment.
   iii. What are the actions the agent can take?
   iv. How can one evaluate the performance of the agent?
   v. What sort of agent architecture do you think is most suitable for this agent? Why?

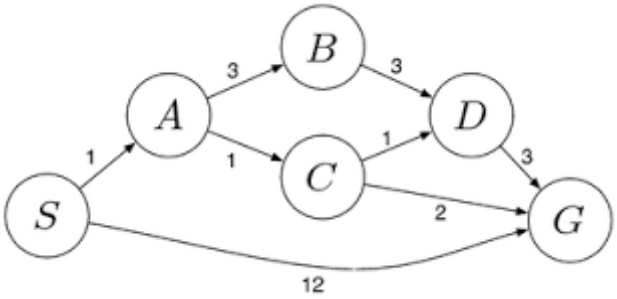
3. Explore the search space using search strategies and formulate the problem components for the 8-Queens problem using the following information: Place 8 queens on a chessboard such that none of the queen's attack any of the others. A configuration of 8 queens on the board as shown in the figure below, but this does not represent a solution as the queen in the first column is on the same diagonal as the queen in the last column.

4. A customer wants to travel from the one location to another location using OLA Cab booking mobile application. The customer can select the any of the cab type as mini, micro, sedan, shared, prime and so on based on his comfort to reach the destination.

Identify what type of problem-solving agent can be used and write the pseudocode for the above problem.

5. A logistics company operates a delivery network where packages must be transported between warehouses at minimum cost. Each route between warehouses has an associated travel cost (fuel + time). The company wants to determine the least-cost path from the starting warehouse **S** to the destination warehouse G using the Uniform Cost Search (UCS) algorithm.

   The delivery network is represented as a weighted graph:



RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process

Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem

1. Ans: **Initial State**

- (4-gallon, 3-gallon) = **(0,0)**

Goal State

- **(2,0)** (Exactly 2 gallons in the 4-gallon jug)

Step	Action	State (4G, 3G)
1	Fill 3-gallon jug	(0,3)
2	Pour 3G into 4G	(3,0)
3	Fill 3-gallon jug again	(3,3)
4	Pour into 4G until full	(4,2)
5	Empty 4-gallon jug	(0,2)
6	Pour remaining 2 gallons into 4G	**(2,0) ✓**

Goal Achieved: Exactly **2 gallons** are in the **4-gallon jug**.

2. Ans: **i) PEAS Description**

Component	Description
Performance Measure	Accurate sample collection, safe navigation, battery efficiency, successful communication
Environment	Mars surface (rocks, sand, hills, unknown terrain, weather)
Actuators	Wheels, robotic arm, drill, camera movement, communication antenna
Sensors	Cameras, spectrometer, GPS/localization sensors, temperature sensors, obstacle sensors

ii) Operating Environment

- Partially observable
- Dynamic
- Sequential
- Stochastic (uncertain)

- Continuous
- Single-agent

iii) Actions

- Move forward/backward
- Turn left/right
- Capture images
- Collect soil/rock samples
- Drill rocks
- Analyze samples
- Send data to Earth
- Avoid obstacles
- Recharge (solar)

iv) Performance Evaluation

- Number of successful samples collected
- Navigation accuracy
- Battery usage
- Distance covered
- Mission completion
- Communication success
- Obstacle avoidance

v) Suitable Agent Architecture

Model-Based Goal-Based Agent

Reason:

- Maintains an internal map of Mars.
- Plans routes toward mission goals.
- Handles unknown environments.
- Makes intelligent decisions using sensor information.

3. Ans: The **8-Queens problem** is a classical Artificial Intelligence (AI) search problem. The objective is to place **eight queens on an 8 × 8 chessboard** such that **no two queens attack each other**. Since a queen can move horizontally, vertically, and diagonally, no two queens should share the same **row, column, or diagonal**. The given configuration is **not a valid solution** because the queen in the **first column** and the queen in the **last column** are positioned on the **same diagonal**, allowing them to attack each other.

The most suitable search strategy is **Depth-First Search (DFS) with Backtracking**.

Justification:

- Places one queen at a time.
- Checks whether the current position is safe.
- If a conflict occurs, the algorithm backtracks to the previous queen and tries another position.
- Efficiently explores the search space while avoiding invalid configurations.

Problem Formulation

- **Initial State:** Empty 8 × 8 chessboard.
- **State:** A partial arrangement of queens on the board.
- **Operators (Actions):** Place one queen in a safe row of the next column.
- **State Space:** All possible arrangements of eight queens on the chessboard.
- **Goal State:** Eight queens are placed so that no two queens attack each other.
- **Path Cost:** Each queen placement has equal cost (1); the aim is to reach any valid solution.

Search Steps

1. Start with an empty chessboard.
2. Place the first queen in the first column.
3. Move to the next column and place another queen in a safe row.
4. Continue placing queens column by column.
5. If no safe position exists in a column, backtrack to the previous column.
6. Repeat until all eight queens are safely placed.

The search space can be explored systematically using **DFS with Backtracking**, which eliminates invalid states and eventually reaches a valid arrangement where:

- No two queens share the same row.
- No two queens share the same column.
- No two queens share the same diagonal.

Thus, the algorithm successfully finds a correct solution to the 8-Queens problem.

The 8-Queens problem demonstrates how **state-space search** and **backtracking** efficiently solve constraint satisfaction problems. Instead of checking every possible arrangement, the algorithm prunes invalid configurations early, reducing computation time. This approach is widely used in Artificial Intelligence for scheduling, planning, constraint satisfaction, and optimization problems.

4. Ans: In the **OLA Cab Booking** application, the customer wants to travel from a **source location** to a **destination**. The application provides different cab options such as **Mini, Micro, Sedan, Prime, Shared**, etc. The customer selects a cab based on comfort, availability, travel time, and fare. The system then chooses the best cab and books it.

Type of Problem-Solving Agent: Goal-Based Agent

Justification:

- The goal is to **reach the destination successfully**.
- The agent evaluates available cab options and selects the one that best satisfies the user's requirements (comfort, fare, availability, and travel time).
- It makes decisions based on achieving the specified goal.

Pseudocode

START

Input Source Location

Input Destination Location

Display available cab types

(Mini, Micro, Sedan, Shared, Prime)

User selects preferred cab type

IF selected cab is available THEN

    Calculate fare

    Display fare and estimated arrival time

    Confirm booking

    Assign nearest driver

    Display booking details

ELSE

    Display "Selected cab not available"

    Suggest other available cab types

ENDIF

END

Input:

- Source location
- Destination location
- Preferred cab type

Processing:

- Check cab availability.
- Calculate fare and estimated arrival time.
- Confirm booking and assign driver.

Output:

- Successful booking details **or**
- Alternative cab options if unavailable.

The **Goal-Based Agent** helps the customer achieve the objective of reaching the destination by selecting the most suitable cab according to the user's preferences and system constraints. This improves decision-making, customer satisfaction, and efficient ride allocation.

5. The logistics network is represented as a **weighted graph**, where:

- **Nodes** represent warehouses.
- **Edges** represent routes between warehouses.
- **Edge weights** represent travel cost (fuel + time).

The objective is to find the **minimum-cost path** from the **source warehouse S** to the **destination warehouse G** using the **Uniform Cost Search (UCS)** algorithm.

Search Algorithm: Uniform Cost Search (UCS)

Justification:

- UCS always expands the node with the **lowest cumulative path cost**.
- It guarantees the **optimal (least-cost) path** when all edge costs are positive.

Step 1: Graph Costs

- S → A = **1**
- S → G = **12**
- A → B = **3**
- A → C = **1**
- B → D = **3**
- C → D = **1**
- C → G = **2**
- D → G = **3**

Step 2: UCS Expansion

Step	Expanded Node	Path Cost	Frontier
1	S	0	A(1), G(12)
2	A	1	C(2), B(4), G(12)
3	C	2	G(4), D(3), B(4), G(12)

Step	Expanded Node	Path Cost	Frontier
4	D	3	G(4), B(4), G(6), G(12)
5	G	**4**	Goal reached

Least-Cost Path

$$\boxed{S \rightarrow A \rightarrow C \rightarrow G}$$

Total Cost

$$1 + 1 + 2 = \boxed{4}$$

Accuracy of Calculation (20%)

Possible paths:

Path	Cost
S → G	12
S → A → B → D → G	1 + 3 + 3 + 3 = **10**
S → A → C → D → G	1 + 1 + 1 + 3 = **6**
S → A → C → G	**1 + 1 + 2 = 4**

The minimum cost is **4**.

Uniform Cost Search explores the graph according to the **lowest accumulated cost**, not the shortest number of edges. It guarantees the **optimal route**.